

Table of Contents

1. [JavaScript Execution Context](#)
 2. [Variable Declarations: var, let, const](#)
 3. [Temporal Dead Zone \(TDZ\)](#)
 4. [Functions in JavaScript](#)
 5. [Function Types and Advanced Concepts](#)
-

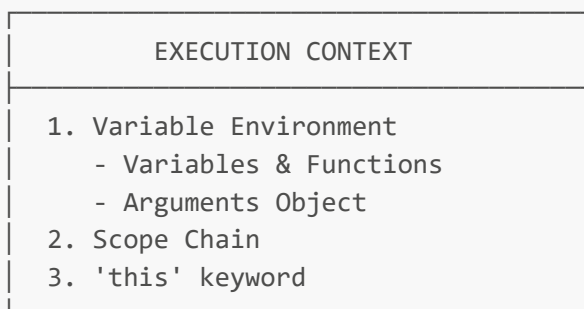
JavaScript Execution Context

Before diving into variables and functions, it's crucial to understand how JavaScript executes code through the **Execution Context**.

What is Execution Context?

Every time JavaScript runs code, it creates an **Execution Context** - a wrapper that contains:

- **Variable Environment** (where variables live)
- **Scope Chain** (access to outer variables)
- **this** keyword



Two Phases of Execution

1. Creation Phase (Variable Phase)

- Memory is allocated for variables and functions
- Variables are set to `undefined`
- Functions are fully loaded into memory

2. Execution Phase

- Code is executed line by line
- Variables are assigned their actual values
- Functions are invoked when called

```
console.log("Start");  
  
console.log(a); // undefined (not error!)  
var a = 10;  
console.log(a); // 10  
  
console.log("End");
```

What happens internally:

```
Creation Phase:  
├─ a: undefined  
└─ console: native function  
  
Execution Phase:  
├─ console.log("Start") → "Start"  
├─ console.log(a) → undefined  
├─ a = 10  
├─ console.log(a) → 10  
└─ console.log("End") → "End"
```

Variable Declarations: var, let, const

JavaScript has three ways to declare variables, each with different behaviors:

var - Function Scoped

```
console.log(x); // undefined (hoisted)
var x = 5;
console.log(x); // 5
```

Characteristics of var :

- **Hoisted** to the top of function/global scope
- **Function-scoped** or **globally-scoped**
- Can be **redeclared**
- **Initialized with** `undefined`

```
function example() {
  console.log(name); // undefined (not error)

  if (true) {
    var name = "JavaScript";
  }

  console.log(name); // "JavaScript" (accessible outside block)
}
```

let - Block Scoped

```
console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 10;
```

Characteristics of `let` :

- **Block-scoped**
- **Hoisted but not initialized** (TDZ)
- **Cannot be redeclared** in same scope
- **Can be reassigned**

```
function example() {  
  // console.log(age); // ReferenceError - TDZ  
  
  if (true) {  
    let age = 25;  
    console.log(age); // 25  
  }  
  
  // console.log(age); // ReferenceError - block scoped  
}
```

`const` - Block Scoped & Immutable

```
const PI = 3.14159;  
// PI = 3.14; // TypeError: Assignment to constant variable
```

Characteristics of `const` :

- **Block-scoped**
- **Hoisted but not initialized** (TDZ)
- **Cannot be redeclared**
- **Cannot be reassigned**
- **Must be initialized** during declaration

```
// const name; // SyntaxError: Missing initializer

const user = {
  name: "John",
  age: 30
};

// user = {}; // TypeError - cannot reassign
user.name = "Jane"; // ☒ OK - can modify object properties
```

Variable Comparison Table

Feature	var	let	const
Scope	Function/Global	Block	Block
Hoisting	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Redeclaration	☒ Allowed	☒ Not Allowed	☒ Not Allowed
Reassignment	☒ Allowed	☒ Allowed	☒ Not Allowed
Initialization	Optional	Optional	Required

Temporal Dead Zone (TDZ)

The **Temporal Dead Zone** is the period between the start of the scope and the actual declaration of `let` / `const` variables.

Understanding TDZ with Examples

```
console.log("Start");

// TDZ for 'b' starts here
console.log(a); // undefined (var hoisting)
// console.log(b); // ReferenceError - b is in TDZ

var a = 10;
let b = 20; // TDZ for 'b' ends here

console.log(a); // 10
console.log(b); // 20
```

TDZ in Practice

```
function example() {
  // TDZ starts for 'temp'
  console.log(temp); // ReferenceError

  let temp = "Hello";
  // TDZ ends for 'temp'

  console.log(temp); // "Hello"
}
```

Complex TDZ Scenario

```
console.log("Start");

var a = 10;

{
  console.log("Inside block");
  // console.log(a); // ReferenceError - 'a' is in TDZ due to let below
  // console.log(b); // ReferenceError - 'b' is in TDZ

  let a = 100; // This creates a new 'a' in block scope
  let b = 200;

  console.log(a); // 100
  console.log(b); // 200
}

console.log("Outside block");
console.log(a); // 10 (global 'a')
// console.log(b); // ReferenceError - 'b' not accessible outside block
```

Memory Representation:

```
Global Execution Context:
├─ var a: 10
└─ block scope:
    ├─ let a: 100 (shadows global 'a')
    └─ let b: 200
```

Functions in JavaScript

Functions are the **heart of JavaScript**. They are **first-class objects**, meaning they can be:

- Stored in variables
- Passed as arguments
- Returned from other functions

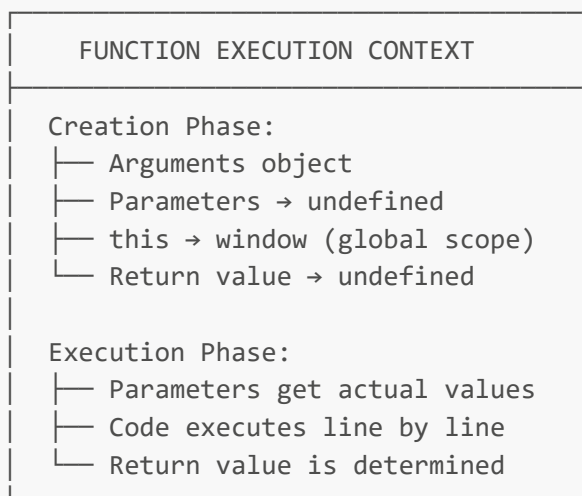
- Have properties and methods

Basic Function Concepts

```
function greet(user) {  
  console.log(`Hello ${user}!`);  
}  
  
greet("JavaScript"); // "Hello JavaScript!"
```

Function Execution Context

When a function is called, JavaScript creates a new **Function Execution Context**:



Function Return Values

Every function returns something - if no explicit return, it returns `undefined`.


```
function sayHello(name) {  
  console.log(`Hello ${name}`);  
  // No explicit return  
}  
  
function calculateSum(a, b) {  
  return a + b; // Explicit return  
}  
  
const result1 = sayHello("World"); // undefined  
const result2 = calculateSum(5, 3); // 8  
  
console.log(result1); // undefined  
console.log(result2); // 8
```

Parameters vs Arguments

```
function sum(a, b) { // 'a' and 'b' are PARAMETERS  
  return a + b;  
}  
  
sum(5, 10); // 5 and 10 are ARGUMENTS
```

Default Parameters

```
function greet(user = "Guest") {  
  console.log(`Hello ${user}!`);  
}  
  
greet(); // "Hello Guest!"  
greet("John"); // "Hello John!"
```

Function Types and Advanced Concepts

1. Function Declaration

```
// Can be called before declaration (hoisting)
sayHello(); // Works!

function sayHello() {
  console.log("Hello World!");
}
```

Hoisting Behavior:

```
console.log(typeof myFunc); // "function" (fully hoisted)

function myFunc() {
  return "I'm hoisted!";
}
```

2. Function Expression

```
// Cannot be called before declaration
// sayHi(); // TypeError: sayHi is not a function

const sayHi = function() {
  console.log("Hi there!");
};

sayHi(); // Works after declaration
```

Hoisting Behavior:

```
console.log(myVar); // undefined
console.log(typeof myVar); // "undefined"

var myVar = function() {
  return "I'm not fully hoisted!";
};
```

3. Arrow Functions (ES6)

```
// Traditional function
const add = function(a, b) {
  return a + b;
};

// Arrow function - explicit return
const addArrow = (a, b) => {
  return a + b;
};

// Arrow function - implicit return
const addShort = (a, b) => a + b;

// Single parameter (parentheses optional)
const double = x => x * 2;

// No parameters
const greet = () => console.log("Hello!");
```

Arrow Function Characteristics:

- No **this** binding (inherits from parent scope)
- No **arguments** object
- Cannot be used as constructors
- Implicit return for single expressions

4. Immediately Invoked Function Expression (IIFE)

```
// Basic IIFE
(function() {
  console.log("I run immediately!");
})();

// IIFE with parameters
(function(name) {
  console.log(`Hello ${name}!`);
})("JavaScript");

// IIFE with return value
const result = (function() {
  return "I'm the result!";
})();

console.log(result); // "I'm the result!"
```

Use Cases for IIFE:

- **Module pattern** (creating private scope)
- **Avoiding global pollution**
- **One-time initialization**

```
const MyModule = (function() {
  let privateVar = 0;

  return {
    increment: function() {
      privateVar++;
    },
    getCount: function() {
      return privateVar;
    }
  };
})();

MyModule.increment();
console.log(MyModule.getCount()); // 1
// console.log(privateVar); // ReferenceError - private!
```

Function Comparison Table

Type	Hoisting	this Binding	arguments	Constructor
Function Declaration	Full	Yes	Yes	Yes
Function Expression	Variable only	Yes	Yes	Yes
Arrow Function	Variable only	No (lexical)	No	No
IIFE	N/A	Yes	Yes	No